Semana 07 — Parchear la causa y probar la re-gresion

Grupo 4 · rol Azul · ejemplo de trabajo

Prof. César Rodríguez | ELECTRONICA | 1 de noviembre de 2026

Curso	CYBERSEGURIDAD — Ciberseguridad Aplicada
Semana	07 de 10
Fase	3 — Defensa y hardening
Grupo	4
Rol	Azul
Entrega	GitHub, carpeta semana_07/
Corte	Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.

1 Tu rol en la práctica

Parte de su propia captura cruda y aplica el control equivalente: configuración, parche o regla de red. Repite el ataque para demostrar que falla y documenta el antes y el después.

- Leer su propia evidencia cruda y elegir el control que corta el ataque en el punto más barato.
- Aplicar el control y repetir el ataque para demostrar que ya no funciona.
- Documentar el antes y el después con los mismos comandos y la diferencia medida.

2 Qué aporta este rol al entregable

La mitigación aplicada, el comando de verificación y la comparación de resultados.

3 Ejemplo paso a paso

El rol Azul escribe el parche en el punto donde nace la falla: parametrizacion, codificacion y lista blanca. Despues lo prueba con el payload del grupo Rojo.

3.1 Parametrizar la consulta

La consulta deja de concatenar y recibe el valor como parametro. El cambio se ve en el diff y se explica en una linea.

```
mkdir -p semana_07/evidencia
git diff -- semana_07/codigo/ > semana_07/evidencia/parche.diff
grep -nE 'execute|cursor' semana_07/codigo/*.py \
  | tee semana_07/evidencia/consulta.txt
```

3.2 Codificar la salida y validar la entrada

La salida se codifica en el contexto donde se usa y la entrada se valida contra un formato esperado. Validar sin codificar no evita el XSS.

3.3 Resolver la ruta con lista blanca

El nombre de archivo se compara contra una lista permitida y se compone con una raiz fija. Concatenar la entrada del usuario y despues "limpiar" no es suficiente.

```
python3 - <<'PY' | tee semana_07/evidencia/rutas.txt
permitidos = {'inicio.php', 'ayuda.php'}
for pedido in ('inicio.php', '../../../../etc/passwd'):
    print(pedido, 'aceptado' if pedido in permitidos else 'rechazado')
PY
```

3.4 Ejecutar la prueba de regresion

La prueba automatizada usa el mismo payload y espera el rechazo. Si la prueba falla, el parche no esta completo.

```
python3 -m pytest semana_07/pruebas -q \
  | tee semana_07/evidencia/pytest.txt
```

3.5 Vulnerabilidad, parche y prueba

Vulnerabilidad, parche y prueba

Vulnerabilidad	Parche	Archivo	Prueba	Resultado
Inyeccion SQL	Consulta parametrizada	consulta.py	test_sqli_rechaza	Pasa
XSS reflejado	Codificacion de salida	vista.py	test_xss_codifica	Pasa
LFI	Lista blanca de rutas	archivos.py	test_lfi_rechaza	Pasa
Regresion	Sin cambios de contrato	consulta.py	test_caso_normal	Pasa

La prueba se cita por su nombre y su resultado; "se probo a mano" no sustituye una prueba reproducible.

4 Qué guardar en el repositorio

Se entrega el diff del parche, la prueba de regresion con su salida y la explicacion de por que la correccion ataca la causa.

```
semana_07/
|-- informe.md
|-- codigo/
|-- pruebas/
|-- evidencia/
|   |-- parche.diff
|   |-- rutas.txt
|   `-- pytest.txt
`-- README.md
```

- El diff muestra el cambio en la causa, no en el mensaje de error.
- La prueba usa el payload original y espera rechazo.
- La regresion funcional sigue en verde.
- La lista blanca se comprueba con el intento de salir de la raiz.

5 Cómo se corrige

Criterio	Peso
Correcciones que eliminan la causa raíz, no el síntoma	3,0
Demostración antes y después con el mismo payload	2,5
Controles automatizados ejecutados e interpretados	2,0
Criterio de aceptación que previene la regresión	1,5
Sin regresiones funcionales en la aplicación	1,0
Total	**10,0**

- Escapar la salida sin validar la entrada deja otras rutas abiertas.
- Una prueba que solo comprueba el caso malo no demuestra que el bueno sigue vivo.

Recuerda

El trabajo es independiente: cada grupo usa su propio repositorio y su propio laboratorio, y la comparación de resultados es en clase. Corte: Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.